

 Example 1 — Detect Specific Bad Domains (Alert + Print)

```
@load base/frameworks/notice # Load the Notice framework so we can generate alerts

# Create a global set (fast lookup list) of known bad domains
global bad_domains = {"bad.com", "evil.com"};

# This event runs every time a DNS request is seen
event dns_request(c: connection, msg: dns_msg, query: string)
{
    # Check if the queried domain exists in our bad_domains set
    if (query in bad_domains)
    {
        # Print message to console/log (for quick visibility/debugging)
        print fmt("Suspicious DNS query: %s", query);

        # Generate an alert using the Notice framework
        NOTICE([
            $note=Notice::PolicyViolation, # Type/category of alert
            $msg=fmt("Detected bad domain: %s", query), # Alert message with domain inserted
            $conn=c # Attach connection info (IP, ports, etc.)
        ]);
    }
}
```

 Example 2 — Detect Possible DNS Tunneling (Long Queries)

```
@load base/frameworks/notice # Load alerting framework

# Triggered on every DNS request
event dns_request(c: connection, msg: dns_msg, query: string)
{
    # Check if the domain length is greater than 50 characters
    if (|query| > 50) # |query| means "length of the string"
    {
        # Print suspicious long query
        print fmt("Long DNS query detected: %s", query);

        # Generate alert for possible DNS tunneling
        NOTICE([
            $note=Notice::PolicyViolation, # Alert type
            $msg=fmt("Possible DNS tunneling: %s", query), # Alert message
            $conn=c # Include connection details
        ]);
    }
}
```

 Example 3 — Track Unique Domains per Host (High Activity Detection)

```
# Create a table:
# Key = IP address (addr)
# Value = set of domains (unique values only)
global dns_tracker: table[addr] of set[string];

# Runs on every DNS request
event dns_request(c: connection, msg: dns_msg, query: string)
{
    # Add the queried domain to this host's set
    dns_tracker[c$Id$orig_h] += query;
    # c$Id$orig_h = source IP address

    # Check how many unique domains this IP has queried
    if (|dns_tracker[c$Id$orig_h]| > 100) # |set| = number of elements
    {
        # Print alert for high DNS activity
        print fmt("High DNS activity from: %s", c$Id$orig_h);
    }
}
```

 Example 4 — Custom DNS Logging (Create Your Own Log File)

```
module DNSLog; # Define a module (namespace) to organize this script

export {
    # Register a new log type called LOG
    redef enum Log::ID += { LOG };

    # Define the structure (columns) of our log
    type Info: record {
        ts: time; # Timestamp of event
        src_ip: addr; # Source IP address
        domain: string; # Queried domain
    };
}

# Runs when Zeek starts
event zeek_init()
{
    # Create the log stream using the structure defined above
    Log::create_stream(LOG, [$columns=Info]);
}

# Runs on every DNS request
event dns_request(c: connection, msg: dns_msg, query: string)
{
    # Write a log entry
    Log::write(LOG, [
        $ts=network_time(), # Current network timestamp
        $src_ip=c$Id$orig_h, # Source IP
        $domain=query # Domain queried
    ]);
}
```

 Example 5 — Detect SSH Brute Force Attempts (Your Custom One)

```
@load base/frameworks/notice # Load alerting framework
@load base/protocols/ssh # Load SSH protocol analyzer

# Define a new Notice type specifically for SSH brute force
redef enum Notice::Type += { SSH_Brute_Force };

# Create a global table:
# Key = IP address
# Value = count of failed SSH attempts
global ssh_attempts: table[addr] of count &default=0;
# &default=0 means every new IP starts with 0 attempts automatically

# This event runs every time an SSH authentication attempt happens
event ssh_auth_attempt(c: connection, success: bool)
{
    # Check if login attempt FAILED
    if (!success) # ! means "NOT" → so this means "if login failed"
    {
        # Increase the failed attempt count for this IP
        ssh_attempts[c$Id$orig_h] += 1;

        # If failed attempts exceed 5
        if (ssh_attempts[c$Id$orig_h] > 5)
        {
            # Generate alert for brute force detection
            NOTICE([
                $note=SSH_Brute_Force, # Custom alert type we defined above

                # Message explaining what happened
                # %s = placeholder for string (IP gets inserted)
                $msg=fmt("Excessive failed SSH attempts from %s",
                    c$Id$orig_h),

                # Attach connection details (VERY IMPORTANT in real-world)
                $conn=c
            ]);
        }
    }
}
```